University of Applied Sciences
Fakultät für Informatik und Wirtschaftsinformatik

Fehlertoleranz und deterministische Zustandsrekonstruktion in MQTT-basierten IIoT- und MES-Architekturen

State of the Art, Fehlermodelle und Architekturvergleich

Exposé und Antrag auf Durchführung einer Masterarbeit
in Kooperation mit der ProSeS GmbH
Master Studiengang Informatik

Autor: Erlind Sejdiu
Matrikelnummer: 106357

Betreuer Hochschule: [Name]
Betreuer Unternehmen: [Name, Abteilung]
Geplanter Zeitraum: [TT.MM.JJJJ] – [TT.MM.JJJJ] (6 Monate)

Stand: 30. Oktober 2026

Inhaltsverzeichnis

1 Motivation **1**

2 Problemstellung **1**

2.1 Was MQTT garantiert und was nicht 1

2.2 Was zustandsbehaftete Prozesse brauchen 1

2.3 Die Lücke 2

3 Forschungsfragen und Ziele **2**

4 Nutzen für die ProSeS GmbH **2**

5 Rahmenbedingungen **3**

6 Zeitplan **3**

Literaturverzeichnis **3**

1 Motivation

Die industrielle Prozessdatenerfassung über Message Queuing Telemetry Transport (MQTT) bildet das nachrichtentechnische Rückgrat moderner Architekturen für das Industrial Internet of Things (IIoT), Fertigungsmanagementsysteme (Manufacturing Execution Systems, MES) sowie Unified Namespaces (UNS) [1]. Maschinen, Steuerungen und Edge-Gateways veröffentlichen ihre Daten auf einem zentralen Broker, und MES, Historian oder Dashboards abonnieren genau die Daten, die sie brauchen. Diese lose Kopplung ist der Grund für die Verbreitung von MQTT, denn Erzeuger und Verbraucher müssen sich nicht kennen [2].

Ein MES arbeitet aber nicht mit einzelnen Messwerten, sondern mit Zuständen. Es muss wissen, welcher Auftrag gerade auf welcher Maschine läuft, wie viele Gutteile gefertigt wurden und ob eine Maschine steht oder produziert [3]. Geht eine Nachricht verloren, kommt sie doppelt oder in falscher Reihenfolge an, dann ist dieser Zustand falsch. Ein verpasstes „Auftrag beendet“ oder ein doppelt gezählter Stückzähler wirkt sich direkt auf Kennzahlen, Rückmeldungen an das ERP-System und Nachweise gegenüber Kunden aus.

Die wissenschaftliche Literatur offenbart eine signifikante Diskrepanz zwischen den elementaren Transportgarantien des MQTT-Standards und den deterministischen Anforderungen zustandsbehafteter Automatisierungsprozesse. MQTT sichert die Zustellung nur zwischen einem Client und dem Broker ab, nicht von Ende zu Ende [4]. Was nach einem Broker-Neustart, einem Netzwerkausfall oder dem Neustart eines Verbrauchers mit dem Zustand passiert, regelt der Standard nicht. Erweiterungen wie Sparkplug B [5] und Muster aus verteilten Systemen wie Event Sourcing [6] oder Log-basierte Plattformen [7] schließen einen Teil dieser Lücke, jeweils mit eigenen Annahmen und Kosten.

Bei der ProSeS GmbH ist MQTT [kurz beschreiben, wo und wie MQTT im Unternehmen eingesetzt wird, z. B. Anbindung von Maschinen an das MES, UNS-Aufbau]. Die Masterarbeit soll klären, wie zuverlässig solche Architekturen den Zustand der Fertigung nach Fehlern wiederherstellen können und welche Architektur dafür am besten geeignet ist.

2 Problemstellung

2.1 Was MQTT garantiert und was nicht

MQTT bietet drei Dienstgüten (Quality of Service, QoS) [4], [8]. QoS 0 stellt höchstens einmal zu, QoS 1 mindestens einmal und QoS 2 genau einmal. Diese Garantien gelten aber jeweils nur für eine Strecke, also vom Publisher zum Broker und getrennt davon vom Broker zum Subscriber. Ob eine Nachricht beim Verbraucher tatsächlich verarbeitet wurde, ist damit nicht gesichert. Die Reihenfolge wird nur innerhalb eines Topics und einer Dienstgüte garantiert, bei Shared Subscriptions in MQTT 5 auch das nicht.

Für den Wiederanlauf gibt es zwei Mechanismen. *Retained Messages* speichern pro Topic genau die letzte Nachricht, und *persistente Sessions* puffern Nachrichten für einen getrennten Client. Beide sind kein Verlauf. Wer nach einem Ausfall wissen will, was in der Zwischenzeit passiert ist, bekommt bestenfalls den letzten Wert, und auch das nur, solange Puffergrenzen und Ablaufzeiten des Brokers nicht überschritten sind. Wie ein Broker seine Daten persistiert und ob er im Cluster betrieben werden kann, ist nicht standardisiert, sondern produktspezifisch.

2.2 Was zustandsbehaftete Prozesse brauchen

Ein MES leitet seinen Zustand aus einer Folge von Ereignissen ab, zum Beispiel Auftragsstart, Statuswechsel und Stückmeldungen. Damit dieser Zustand nach einem Fehler korrekt wiederhergestellt werden kann, muss er *deterministisch* aus den Ereignissen folgen, also bei gleicher Eingabe immer gleich sein [9]. Dafür braucht es eine eindeutige Reihenfolge der

Ereignisse [10], eine idempotente Verarbeitung von Duplikaten und einen Punkt, ab dem nach einem Ausfall weitergemacht werden kann [11], [12].

2.3 Die Lücke

Zwischen diesen beiden Seiten gibt es eine Lücke, die in der Praxis meist anwendungsspezifisch und oft ungeprüft geschlossen wird. Es fehlt eine zusammenhängende Darstellung,

- welche Fehler in MQTT-basierten IIoT- und MES-Architekturen tatsächlich auftreten,
- welche davon von MQTT, Sparkplug B oder dem Broker abgefangen werden und welche nicht,
- und welche Architektur unter welchen Fehlern den Zustand korrekt rekonstruiert.

Abgrenzung. Die Arbeit betrachtet Fehlertoleranz und Zustandsrekonstruktion. IT-Sicherheit (Authentifizierung, Verschlüsselung, Angriffe) ist nicht Gegenstand der Arbeit, ebenso wenig harte Echtzeitanforderungen auf Feldebene. Andere Protokolle wie OPC UA werden nur als Quelle der Daten berücksichtigt, nicht als Alternative zu MQTT verglichen. Es wird kein neues MES entwickelt. Die Untersuchung erfolgt in einer Testumgebung und nicht im laufenden Produktivbetrieb.

3 Forschungsfragen und Ziele

Ziel der Arbeit ist es, den Stand der Technik zur Fehlertoleranz in MQTT-basierten Architekturen aufzuarbeiten, die relevanten Fehler in einem Fehlermodell festzuhalten und verbreitete Architekturen daran zu messen. Daraus ergeben sich drei Forschungsfragen.

FF1 Welche Ansätze zur Fehlertoleranz und Zustandsrekonstruktion in MQTT-basierten IIoT- und MES-Architekturen beschreibt die Literatur, und welche Garantien bieten sie?
FF2 Welche Fehler treten in solchen Architekturen auf, und welche Auswirkung haben sie auf den rekonstruierten Zustand eines MES?
FF3 Wie gut rekonstruieren verbreitete Architekturvarianten den Zustand unter diesen Fehlern, und mit welchem Aufwand?

Die Arbeit liefert drei Ergebnisse. Erstens eine strukturierte Übersicht über den Stand der Technik und verwandte Arbeiten (FF1). Zweitens ein Fehlermodell, das Fehlerorte, Fehlerarten und ihre Auswirkung auf den Zustand beschreibt (FF2). Drittens einen Architekturvergleich mit Messergebnissen aus einer Testumgebung und einer Empfehlung für die ProSeS GmbH (FF3).

4 Nutzen für die ProSeS GmbH

Die Arbeit liefert der ProSeS GmbH eine belastbare Grundlage für Architekturentscheidungen rund um MQTT, UNS und MES. Im Einzelnen:

- ein Fehlermodell, mit dem sich bestehende und geplante Anlagen systematisch auf Schwachstellen prüfen lassen,
- einen messbasierten Vergleich von Architekturvarianten statt einer Entscheidung nach Erfahrung oder Herstellerangaben,
- eine wiederverwendbare Testumgebung, mit der sich Broker, Konfigurationen und Updates vor dem Einsatz unter Fehlerbedingungen testen lassen,
- konkrete Handlungsempfehlungen, zum Beispiel zu QoS-Einstellungen, Session-Konfiguration, Puffergrößen und Idempotenz in Verbrauchern.

Gerade weil Fehler im Datenfluss in der Fertigung oft erst spät bemerkt werden, etwa als falsche Kennzahl oder fehlende Rückmeldung, ist ein systematischer Blick auf diese Fehler

auch für den Betrieb wertvoll.

5 Rahmenbedingungen

Unterstützung durch das Unternehmen. Für die Durchführung wäre Folgendes hilfreich:

- eine Ansprechperson als Betreuung im Unternehmen und regelmäßige kurze Abstimmungen,
- Einblick in die eingesetzte Architektur (Broker-Produkt, Topic-Struktur, Datenmodell, Verbraucher),
- wenn möglich anonymisierte Beispieldaten oder Logs zu bisherigen Störungen, um das Fehlermodell an realen Fällen zu prüfen,
- Zugang zu einer Test- oder Entwicklungsumgebung bzw. die Freigabe, Broker-Software in einer eigenen Umgebung zu nutzen.

Kein Eingriff in die Produktion. Alle Fehlerexperimente finden ausschließlich in einer isolierten Testumgebung statt. Produktivsysteme werden nicht verändert.

Vertraulichkeit. Unternehmensinterne Informationen werden nur in Abstimmung mit der ProSeS GmbH verwendet. Bei Bedarf wird die Arbeit mit einem Sperrvermerk versehen. Die wissenschaftlichen Ergebnisse, also Literaturübersicht, Fehlermodell und Vergleich, werden so formuliert, dass sie ohne interne Details nachvollziehbar sind.

6 Zeitplan

Die Bearbeitungszeit beträgt sechs Monate.

Phase	Monat	Ziel	Artefakt
1 Einarbeitung	1	Bestehende Architektur bei der ProSeS GmbH verstehen, Recherche planen	Recherche-Protokoll
2 State of the Art	1–2	Literatur und Standards auswerten	Literaturübersicht (FF1)
3 Fehlermodell	2–3	Fehlerorte, Fehlerarten und Auswirkungen festhalten	Fehlermodell (FF2)
4 Testumgebung	3–4	Architekturvarianten und Fehlerinjektion aufbauen	Lauffähige Testumgebung
5 Experimente	4–5	Fehlerfälle durchspielen und messen	Messergebnisse (FF3)
6 Auswertung	5	Vergleich und Empfehlung ableiten	Bewertung, Empfehlung
7 Ausarbeitung	5–6	Schriftliche Arbeit und Abschlusspräsentation	Masterarbeit, Code im Repository

Literaturverzeichnis

[1] B. Mishra und A. Kertesz, „The Use of MQTT in M2M and IoT Systems: A Survey,“ *IEEE Access*, Jg. 8, S. 201 071–201 086, 2020. DOI: 10.1109/ACCESS.2020.3035849

[2] P. T. Eugster, P. A. Felber, R. Guerraoui und A.-M. Kermarrec, „The Many Faces of Publish/Subscribe,“ *ACM Computing Surveys*, Jg. 35, Nr. 2, S. 114–131, 2003. DOI: 10.1145/857076.857078

[3] *IEC 62264-1:2013 – Enterprise-control system integration – Part 1: Models and terminology*, International Electrotechnical Commission, 2013.

[4] A. Banks, E. Briggs, K. Borgendale und R. Gupta, Hrsg., *MQTT Version 5.0*, OASIS Standard, OASIS, 2019. besucht am 27. Okt. 2026. Adresse: https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html

[5] *Sparkplug Specification, Version 3.0.0*, Eclipse Foundation, 2022. besucht am 27. Okt. 2026. Adresse: https://sparkplug.eclipse.org/specification/

[6] M. Fowler. „Event Sourcing," besucht am 27. Okt. 2026. Adresse: https://martinfowler.com/eaaDev/EventSourcing.html

[7] J. Kreps, N. Narkhede und J. Rao, „Kafka: A Distributed Messaging System for Log Processing," in *Proceedings of the 6th International Workshop on Networking Meets Databases (NetDB)*, 2011.

[8] A. Banks und R. Gupta, Hrsg., *MQTT Version 3.1.1*, OASIS Standard, OASIS, 2014. besucht am 27. Okt. 2026. Adresse: https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html

[9] F. B. Schneider, „Implementing Fault-Tolerant Services Using the State Machine Approach: A Tutorial," *ACM Computing Surveys*, Jg. 22, Nr. 4, S. 299–319, 1990. DOI: 10.1145/98163.98167

[10] L. Lamport, „Time, Clocks, and the Ordering of Events in a Distributed System," *Communications of the ACM*, Jg. 21, Nr. 7, S. 558–565, 1978. DOI: 10.1145/359545.359563

[11] E. N. Elnozahy, L. Alvisi, Y.-M. Wang und D. B. Johnson, „A Survey of Rollback-Recovery Protocols in Message-Passing Systems," *ACM Computing Surveys*, Jg. 34, Nr. 3, S. 375–408, 2002. DOI: 10.1145/568522.568525

[12] K. M. Chandy und L. Lamport, „Distributed Snapshots: Determining Global States of Distributed Systems," *ACM Transactions on Computer Systems*, Jg. 3, Nr. 1, S. 63–75, 1985. DOI: 10.1145/214451.214456